Customer Churn Analysis (ANN Model)
ACS WILDA Team 7

Team Member

1. Prashant Raj Shrestha
2. Sadaf Mehedi Chyon
3. Prakhar Tewari
4. Yishi Liu
5. Riyaben Patel

Predictive Modelling Documentation

Import Required Libraries

```
#Import required libraries
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, classification_report, roc_curve, auc
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
```

The first step is to import all the required libraries. For handling and manipulating the dataset, **Pandas** library is used. The libraries of **TensorFlow** and **Keras** are used to built and train the Artificial Neural Network (ANN). Specific keras layers dense, dropout and batch normalization are imported to construct the ANN architecture.

The **Sklearn** library is used to split the data into training and testing. It also does feature scaling and helps with confusion matrix, classification report, ROC curve and AUC score. For plotting the graphs and data visualisation the libraries used are **matplotlib** and **seaborn**.

Load Dataset

```
# Load dataset
df = pd.read_csv("preprocessed_full_dataset.csv")
df.head()
```

	gender	SeniorCitizen	Dependents	tenure	PhoneService	MultipleLines	MonthlyCharges	Churn	InternetService_Fiber optic	Contract_One year	Contract_Two year
0	0	0	0	-1.277445	0	0	-1.321350	1	False	False	False
1	1	0	0	0.351370	1	0	-1.321350	0	False	True	False
2	0	0	1	0.799294	1	0	-1.520755	0	False	False	False
3	0	0	0	-1.277445	1	0	0.373593	1	False	True	False
4	1	0	0	1.410099	1	0	-0.457261	0	True	False	False

The dataset is loaded to the environment using the read_csv() function from pandas. This dataset has customer related attributes along with a target variable which indicates weather a customer has churned or not. The dataset is pre-processed and appropriately structured so that it is ready for the predictive modelling.

The df.head() function is used to display the first five rows of our dataset which gives a small preview of the dataset. It has columns like tenure, MonthlyCharges and Churn which are important for predictive modelling.

Separate Features and Target Variables

```python
# Separate features (X) and target (y)
X = df.drop(columns=["Churn"])   # all columns except target
y = df["Churn"]                  # target variable
print(X)
print(y)
```

In this step, the dataset is divided into input features and an output target variable. The input **X** contains all independent variables and the target variable **y** contains the churn values.

```
      gender  SeniorCitizen  Dependents    tenure  PhoneService  \
0          0              0           0 -1.277445             0
1          1              0           0  0.351370             1
2          0              0           1  0.799294             1
3          0              0           0 -1.277445             1
4          1              0           0  1.410099             1
...      ...            ...         ...       ...           ...
7038       1              0           0 -1.277445             1
7039       0              0           1 -0.381597             1
7040       1              0           1 -0.829521             1
7041       1              1           0 -0.829521             1
7042       1              0           0 -0.259435             1

      MultipleLines  MonthlyCharges  InternetService_Fiber optic  \
0                 0       -1.321350                       False
1                 0       -1.321350                       False
2                 0       -1.520755                       False
3                 0        0.373593                       False
4                 0       -0.457261                        True
...             ...             ...                         ...
7038              1        1.005042                       False
7039              1        0.872106                        True
7040              0       -1.454286                       False
7041              1        1.137979                        True
7042              0       -1.487520                        True

      Contract_One year  Contract_Two year
0                 False              False
1                  True              False
2                 False              False
3                  True              False
4                 False              False
...                 ...                ...
7038               True              False
7039               True              False
7040              False              False
7041              False              False
7042              False               True

[7043 rows x 10 columns]
```

```
0       1
1       0
2       0
3       1
4       0
       ..
7038    1
7039    0
7040    0
7041    1
7042    0
Name: Churn, Length: 7043, dtype: int64
```

It is important to separate because the model learns from input features to predict the output. By separating the churn column from the dataset, the model focuses on the relationship between the attributes and the churn behaviour.

Training and Testing Data

The dataset is split into training and testing data using a ratio of 80:20 means 80% data is training and 20% is testing. The training set is used to train the model, and the testing set is used for performance evaluation. Even though the dataset was pre-processed, the train-test split was done to ensure that the model does not overfit and gives a realistic measure on how the model will perform in real-world situations.

```
# Perform Train-Test Split (80-20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y   # IMPORTANT: keeps churn distribution balanced
)
```

The **stratify=y** is used to make sure that the distribution of churn and non-churn customers remained consistent across both training and testing as the dataset is imbalanced as we have 73% churned customers and 27% of non-churned customers.

```
# Check shapes
print("Training set:", X_train.shape)
print("Testing set:", X_test.shape)
```

```
Training set: (5634, 10)
Testing set: (1409, 10)
```

The shapes are printed to confirm that training and testing are performed successfully. Training set has 5634 samples while testing has 1409 samples.

```python
# Check distribution
#Training
print("\nChurn distribution in training set:")
print(y_train.value_counts(normalize=True))
#Testing
print("\nChurn distribution in testing set:")
print(y_test.value_counts(normalize=True))
```

```
Churn distribution in training set:
Churn
0    0.734647
1    0.265353
Name: proportion, dtype: float64

Churn distribution in testing set:
Churn
0    0.734564
1    0.265436
Name: proportion, dtype: float64
```

By checking class distribution, we can see that the data is imbalance between churn and non-churn customers. It motivates using the techniques like class weightage for model training.

```python
# Combine X and y back together for saving
train_data = X_train.copy()
train_data["Churn"] = y_train
test_data = X_test.copy()
test_data["Churn"] = y_test

# Save to CSV files
train_data.to_csv("train_data.csv", index=False)
test_data.to_csv("test_data.csv", index=False)

print("Train and Test datasets saved successfully!")
```

```
Train and Test datasets saved successfully!
```

The X and y are combined together for the purpose of saving the data. Training and testing data are saved in separate csv files.

Feature Scaling

```python
# Feature Scaling

# Create scaler object
scaler = StandardScaler()
# Columns to scale
num_cols = ["tenure", "MonthlyCharges"]

# Fit ONLY on training data, transform both
X_train[num_cols] = scaler.fit_transform(X_train[num_cols])
X_test[num_cols] = scaler.transform(X_test[num_cols])

#Scaling numerical values
print(X_train[["tenure", "MonthlyCharges"]].head())
print()
print("Means:")
print(X_train[["tenure", "MonthlyCharges"]].mean())
print()
print("Standard deviations:")
print(X_train[["tenure", "MonthlyCharges"]].std())
```

Feature Scaling is done to variables tenure and MonthlyCharges, which are numerical values, using the **StandardScaler**. This is done to convert the features so that their mean is 0 and standard deviation is 1. As the neural networks are sensitive to scale of input features,

scaling is important. Without it, the input features which has larger values can dominate the learning of the model. By checking the results, we can see that mean is closer to 0 while the standard deviation is 1.

ANN Model Architecture

```
# Build ANN model
model = Sequential([
    Dense(256, activation="relu", input_shape=(X_train.shape[1],)),
    BatchNormalization(),
    Dropout(0.3),

    Dense(128, activation="relu"),
    BatchNormalization(),
    Dropout(0.25),

    Dense(64, activation="relu"),
    BatchNormalization(),
    Dropout(0.2),

    Dense(16, activation="relu"),
    Dropout(0.15),

    Dense(1, activation="sigmoid")
])
```

The ANN model is made using the sequential architecture. The model has 1 input layer, four hidden layers and 1 output layer. The hidden layers contains 256, 128, 64 and 16 neurons respectively. ReLU activation is used for non-linearity. To stabilize and accelerate the training, batch normalisation layers are added. Dropout layers are used to reduce the model overfitting. Sigmoid activation is used for output layer which has probability between 0 and 1 which is suitable for binary classification.

```
# Compile model
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

```
# Show model summary
model.summary()
```

The model is compiled using the Adam optimizer. For loss binary cross entropy is used as the problem is a binary classification task. Accuray is used as performance metric.

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_5 (Dense)	(None, 256)	2,816
batch_normalization_3 (BatchNormalization)	(None, 256)	1,024
dropout_4 (Dropout)	(None, 256)	0
dense_6 (Dense)	(None, 128)	32,896
batch_normalization_4 (BatchNormalization)	(None, 128)	512
dropout_5 (Dropout)	(None, 128)	0
dense_7 (Dense)	(None, 64)	8,256
batch_normalization_5 (BatchNormalization)	(None, 64)	256
dropout_6 (Dropout)	(None, 64)	0
dense_8 (Dense)	(None, 16)	1,040
dropout_7 (Dropout)	(None, 16)	0
dense_9 (Dense)	(None, 1)	17

Total params: 46,817 (182.88 KB)
Trainable params: 45,921 (179.38 KB)
Non-trainable params: 896 (3.50 KB)

The model summary provides details about all the layers including output shape and parameters. Total number of parameters are 46,817 out of which 45,921 parameters are trainable while 896 are non-trainable.

```
print(X_train.shape)
print(X_train.columns)
```

```
(5634, 10)
Index(['gender', 'SeniorCitizen', 'Dependents', 'tenure', 'PhoneService',
       'MultipleLines', 'MonthlyCharges', 'InternetService_Fiber optic',
       'Contract_One year', 'Contract_Two year'],
      dtype='object')
```

A feature verification is done to determine whether correct variables are used or not. This is the final verification before the model training.

```
#Train the model

history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2,
    class_weight={0:1, 1:2}
)
```

For the model training, 50 epochs is used with a batch size of 32. For monitoring the performance, a validation split of 20% is used. Class weightage is applied so that the model can learn better from the imbalance data.

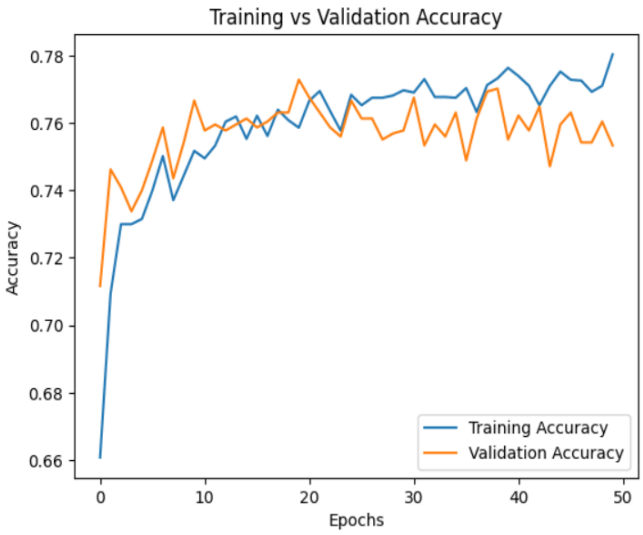
```
Epoch 1/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - accuracy: 0.6607 - loss: 0.8353 - val_accuracy: 0.7116 - val_loss: 0.6244
Epoch 2/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.7093 - loss: 0.7286 - val_accuracy: 0.7462 - val_loss: 0.5470
Epoch 3/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.7300 - loss: 0.7102 - val_accuracy: 0.7409 - val_loss: 0.4946
Epoch 4/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.7300 - loss: 0.6930 - val_accuracy: 0.7338 - val_loss: 0.4817
Epoch 5/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.7315 - loss: 0.6938 - val_accuracy: 0.7400 - val_loss: 0.4855
Epoch 6/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.7400 - loss: 0.6903 - val_accuracy: 0.7489 - val_loss: 0.4779
Epoch 7/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.7502 - loss: 0.6831 - val_accuracy: 0.7587 - val_loss: 0.4659
Epoch 8/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.7371 - loss: 0.6803 - val_accuracy: 0.7436 - val_loss: 0.4760
Epoch 9/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.7444 - loss: 0.6878 - val_accuracy: 0.7542 - val_loss: 0.4659
Epoch 10/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.7517 - loss: 0.6735 - val_accuracy: 0.7666 - val_loss: 0.4664
Epoch 11/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.7495 - loss: 0.6785 - val_accuracy: 0.7578 - val_loss: 0.4746
Epoch 12/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.7533 - loss: 0.6711 - val_accuracy: 0.7595 - val_loss: 0.4621
Epoch 13/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.7604 - loss: 0.6778 - val_accuracy: 0.7578 - val_loss: 0.4642
Epoch 14/50
141/141 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.7619 - loss: 0.6648 - val_accuracy: 0.7595 - val_loss: 0.4676
Epoch 15/50
```

This shows that as the training progress, the model improves and stabilizes, which indicates that the learning has been successful.

Evaluate Model Performance

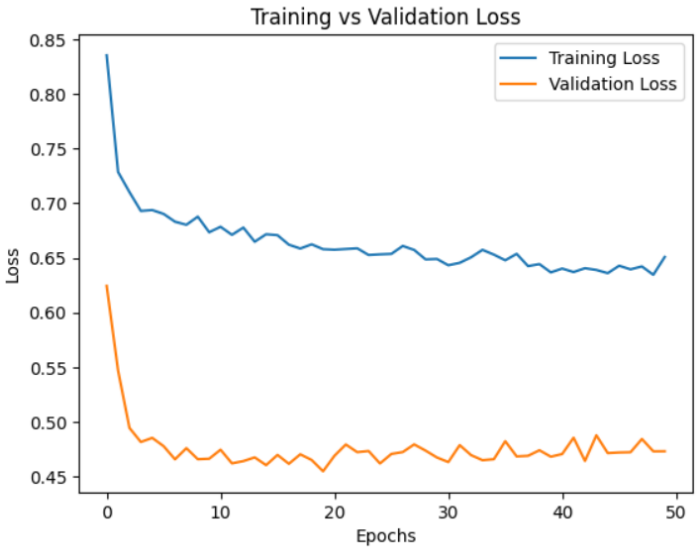
```
#Training accuracy vs Validation accuracy


plt.figure()
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Training vs Validation Accuracy')
plt.legend()
plt.show()
```



The graph compares training and validation performance over the epochs. Both the curves remain close to each other, with only minor fluctuations. This is the indication that the model has generalized well and there was no overfitting.

```
#Loss curve plot
plt.figure()
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Training vs Validation Loss')
plt.legend()
plt.show()
```



The graph of training vs validation loss shows a steady decrease in training loss while validation loss is stable. It confirms that without being overfitting, the model is learning well.

```
#Evaluate test data
test_loss, test_accuracy = model.evaluate(X_test, y_test)
print("Test Accuracy:", test_accuracy)
print("Test Loss:", test_loss)
```

```
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.7537 - loss: 0.4986
Test Accuracy: 0.753726065158844
Test Loss: 0.498647004365921
```

```
#Model Performance
y_pred_prob = model.predict(X_test)
y_pred = (y_pred_prob > 0.55).astype(int)

print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step
[[855 180]
 [152 222]]
              precision    recall  f1-score   support

           0       0.85      0.83      0.84      1035
           1       0.55      0.59      0.57       374

    accuracy                           0.76      1409
   macro avg       0.70      0.71      0.70      1409
weighted avg       0.77      0.76      0.77      1409
```

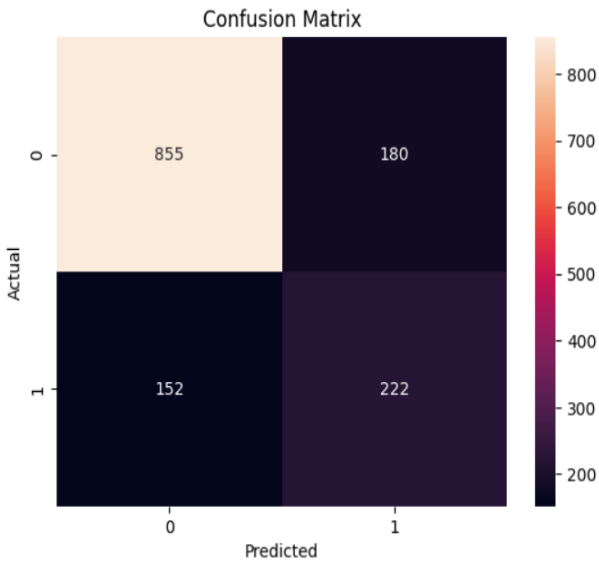
The model is evaluated on the test data, and it shows an accuracy of around 75%. Model prediction is done on a probability of 0.55 instead of default 0.5. This helps in balancing precision and recall. It is an improvement from the default probability of 0.5. Model accuracy is 76%, churn recall is 62%, precision is 53% and F1-score is 57%.

The confusion matrix provides a complete breakdown of the outcome. The model had identified many customers who are non-churn. Even though there seems to be some

misclassification, the balance model achieves between false positives and false negatives is good.

```
#Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

plt.figure()
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()
```

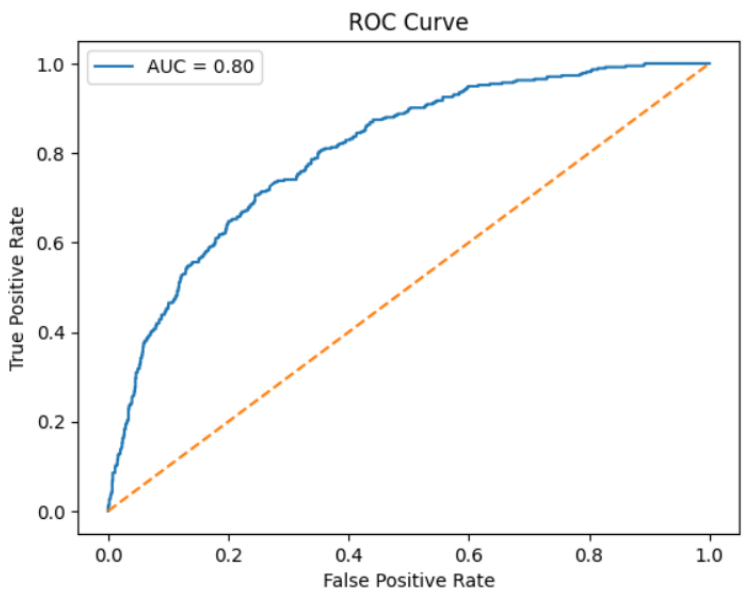


True negatives are 855, the number of customers who will stay, correctly predicted by the model. False positives are 180, model incorrectly predicted as churned. False negatives are 152, the number of churn customers missed by the model. True positives are 222, the number of customers model correctly identified as churn customers.

```
#ROC Curve


fpr, tpr, _ = roc_curve(y_test, y_pred_prob)
roc_auc = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr, label='AUC = %0.2f' % roc_auc)
plt.plot([0, 1], [0, 1], linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.show()
```



The ROC curve shows the trade-off between true positive rate and false positive rate across the thresholds. The model achieved an AUC of 0.80. It indicates good classification performance. Since the curve is above the diagonal, it means that the model performs much better than the random guessing.

Save Model

```
#Save Model
model.save("churn_ann_model.keras")
print("Model saved successfully")
```

```
Model saved successfully
```

After training and evaluating the model, last step is to save the trained model so that it can be reused for future predictions. The model was saved using TensorFlow's in built function model.save(). The model was saved in native keras format instead of using .h5 format which is an older version.